

C++ Standard Library Issues to be moved in Kona, Nov. 2023

Doc. no. P3040R0

Date: 2023-11-07

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

- [Ready Issues](#)
- [Tentatively Ready Issues](#)

Ready Issues

[2392](#). "character type" is used but not defined

Section: 3.36 [\[defns.ntcts\]](#), 30.3.1.2.1 [\[locale.category\]](#), 31.2.3 [\[iostreams.limits.pos\]](#), 31.7.6.3.1 [\[ostream.formatted.reqmts\]](#), 31.7.6.3.4 [\[ostream.inserters.character\]](#) **Status:** [Ready](#) **Submitter:** Jeffrey Yasskin **Opened:** 2014-06-01 **Last modified:** 2023-06-26

Priority: 3

Discussion:

The term "character type" is used in 3.36 [\[defns.ntcts\]](#), 30.3.1.2.1 [\[locale.category\]](#), 31.2.3 [\[iostreams.limits.pos\]](#), 31.7.6.3.1 [\[ostream.formatted.reqmts\]](#), and 31.7.6.3.4 [\[ostream.inserters.character\]](#), but the core language only defines "narrow character types" (6.8.2 [\[basic.fundamental\]](#)).

"wide-character type" is used in D.26 [\[depr.locale.stdcvt\]](#), but the core language only defines a "wide-character set" and "wide-character literal".

[2023-06-14; Varna; Daniel comments and provides wording]

Given the resolution of [P2314](#) which had introduced to 6.8.2 [\[basic.fundamental\]](#) p11 a definition of "character type":

The types `char`, `wchar_t`, `char8_t`, `char16_t`, `char32_t` are collectively called *character types*.

one might feel tempted to have most parts of this issue resolved here, but I think that this actually is a red herring.

First, as Jonathan already pointed out, for two places, 31.7.6.3.1 [\[ostream.formatted.reqmts\]](#) and 31.7.6.3.4 [\[ostream.inserters.character\]](#), this clearly doesn't work, instead it seems as if we should replace "character type of the stream" here by "char_type of the stream".

To me "char_type of the stream" sounds a bit odd (we usually refer to `char_type` in terms of a qualified name such as `X::char_type` instead unless we are specifying a member of some `X`, where we can omit the qualification) and in the suggested wording below I'm taking advantage of the already defined term "character container type" (3.10 [\[defns.character.container\]](#)) instead, which seems to fit its intended purpose here.

Second, on further inspection it turns out that actually only *one* usage of the term "character type" seems to be intended to refer to the actual core language meaning (See the unchanged wording for 30.4.3.3.3 [\[facet.num.put.virtuals\]](#) in the proposed wording below), all other places quite clearly must refer to the above mentioned "character container type".

For the problem related to the missing definition of "wide-character type" (used two times in D.26 [\[depr.locale.stdcvt\]](#)) I would like to suggest a less general and less inventive approach to solve the definition problem here, because it only occurs in an already deprecated component specification: My suggestion is to simply get rid of that term by just identifying `Elem` with being one of `wchar_t`, `char16_t`, or `char32_t`. (This result is identical to identifying "wide-character type" with a "character type that is not a narrow character type" (6.8.2 [\[basic.fundamental\]](#))), but this seemingly more general definition doesn't provide a real advantage.)

[Varna 2023-06-14; Move to Ready]

[2023-06-25; Daniel comments]

During the Varna LWG discussions of this issue it had been pointed out that the wording change applied to D.26.3 [\[depr.locale.stdcvt.req\]](#) bullet (1.1) could exclude now the previously allowed support of narrow character types as a "wide-character" with e.g. a `Maxcode` value of 255. First, I don't think that the revised wording really forbids this. Second, the originating proposal [N2401](#) doesn't indicate what the actual intend here was and it seems questionable to assign LEWG to this issue given that the relevant wording is part of deprecated components, especially given their current position expressed [here](#) to eliminate the specification of the affected components as suggested by [P2871](#).

Proposed resolution:

This wording is relative to [N4950](#).

[Drafting note: All usages of "character type" in 22.14 [\[format\]](#) seem to be without problems.]

1. Modify 30.3.1.2.1 [\[locale.category\]](#) as indicated:

[Drafting note: The more general interpretation of "character container type" instead of character type by the meaning of the core language seems safe here. It seems reasonable that an implementation allows more than the core language character types, but still could impose additional constraints imposed on them. Even if an implementation does never intend to support anything beyond char and wchar_t, the wording below is harmless. One alternative could be here to use the even more general term "char-like types" from 23.1 [\[strings.general\]](#), but I'm unconvinced that this buys us much]

-6- [...] A template parameter with name C represents the set of types containing char, wchar_t, and any other implementation-defined character [container](#) types ([3.10 \[defns.character.container\]](#)) that meet the requirements for a character on which any of the istream components can be instantiated. [...]

2. Keep 30.4.3.3.3 [\[facet.num.put.virtuals\]](#) of **Stage 1** following p4 *unchanged*:

[Drafting note: The wording here seems to refer to the pure core language wording meaning of a character type.]

[...] For conversion from an integral type other than a character type, the function determines the integral conversion specifier as indicated in Table 110.

3. Modify 31.2.3 [\[iostreams.limits.pos\]](#) as indicated:

[Drafting note: Similar to 30.3.1.2.1 [\[locale.category\]](#) above the more general interpretation of "character container type" instead of character type by the meaning of the core language seems safe here.]

-3- In the classes of Clause 31, a template parameter with name charT represents a member of the set of types containing char, wchar_t, and any other implementation-defined character [container](#) types ([3.10 \[defns.character.container\]](#)) that meet the requirements for a character on which any of the istream components can be instantiated.

4. Modify 31.7.6.3.1 [\[ostream.formatted.reqmts\]](#) as indicated:

-3- If a formatted output function of a stream os determines padding, it does so as follows. Given a charT character sequence seq where charT is the character [container](#) type of the stream, [...]

5. Modify 31.7.6.3.4 [\[ostream.inserters.character\]](#) as indicated:

```
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, charT c);
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, char c);
// specialization
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, char c);
// signed and unsigned
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, signed char c);
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, unsigned char c);
```

-1- *Effects*: Behaves as a formatted output function (31.7.6.3.1 [\[ostream.formatted.reqmts\]](#)) of out. Constructs a character sequence seq. If c has type char and the character [container](#) type of the stream is not char, then seq consists of out.widen(c); otherwise seq consists of c. Determines padding for seq as described in 31.7.6.3.1 [\[ostream.formatted.reqmts\]](#). Inserts seq into out. Calls os.width(0).

6. Modify D.26.3 [\[depr.locale.stdcvt.req\]](#) as indicated:

(1.1) — Elem is [one of the wide-character type, such as](#) wchar_t, char16_t, or char32_t.

(1.2) — Maxcode is the largest [wide-character code value of Elem converted to unsigned long](#) that the facet will read or write without reporting a conversion error.

[...]

Section: 24.7.2.2.1 [\[span.overview\]](#) Status: [Ready](#) Submitter: Johel Ernesto Guerrero Peña Opened: 2019-05-04 Last modified: 2023-06-14

Priority: 2

View other [active issues](#) in [\[span.overview\]](#).

View all other [issues](#) in [\[span.overview\]](#).

Discussion:

span doesn't explicitly point out when its accessed elements are invalidated like string_view does in 23.3.3.4 [\[string.view.iterators\]](#) p2.

[2019-06-12 Priority set to 2 after reflector discussion]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 24.7.2.2.1 [\[span.overview\]](#) as indicated:

-4- ElementType is required to be a complete object type that is not an abstract class type.

~~For a span s, any operation that invalidates a pointer in the range [s.data(), s.data() + s.size()).~~
~~invalidates pointers, iterators, and references other than *this returned from s's member functions.~~

[2023-06-13; Varna]

The reference to [N4910](#) 23.3.3.4 [\[string.view.iterators\]](#) p2 is located now in [N4950](#) 23.3.3.1 [\[string.view.template.general\]](#) p2 where its says:

For a basic_string_view str, any operation that invalidates a pointer in the range

[str.data(), str.data() + str.size())

invalidates pointers, iterators, and references returned from str's member functions.

The group also suggested to adjust 23.3.3.1 [\[string.view.template.general\]](#) p2 similarly.

[2023-06-14 Varna; Move to Ready]

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 23.3.3.1 [\[string.view.template.general\]](#) as indicated:

[Drafting note: The proposed wording also removes the extra code block that previously defined the range]

~~-2- For a basic_string_view str, any operation that invalidates a pointer in the range [str.data(), str.data() + str.size()) invalidates pointers, iterators, and references~~
~~to elements of str returned from str's member functions.~~

2. Modify 24.7.2.2.1 [\[span.overview\]](#) as indicated:

-4- ElementType is required to be a complete object type that is not an abstract class type.

~~For a span s, any operation that invalidates a pointer in the range [s.data(), s.data() + s.size()) invalidates pointers,~~
~~iterators, and references to elements of s.~~

[3305](#). any_cast<void>

Section: 22.7.5 [\[any.nonmembers\]](#) Status: [Ready](#) Submitter: John Shaw Opened: 2019-10-16 Last modified: 2023-06-14

Priority: 2

View other [active issues](#) in [\[any.nonmembers\]](#).

View all other [issues](#) in [\[any.nonmembers\]](#).

Discussion:

```
any foo;
void* p = any_cast<void>(&foo);
```

Per 22.7.5 [\[any.nonmembers\]](#)/9, since the operand isn't nullptr and operand->type() == typeid(T) (because T = void in this case), we should return a pointer to the object contained by operand. But there is no such object.

We need to handle the T = void case, probably by just explicitly returning nullptr.

[2019-11 Priority to 2 during Monday issue prioritization in Belfast. There is implementation divergence here.]

[2020-02 LWG discussion in Prague did not reach consensus. Status to Open.]

There was discussion about whether or not any_cast<void>(a) should be ill-formed, or return nullptr.

Poll "should it return nullptr" was 0-4-5-5-1.

[2022-02 Currently ill-formed in MSVC ("error C2338: std::any cannot contain void") and returns null pointer in libstdc++ and libc++.]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4835](#).

1. Modify 22.7.5 [\[any.nonmembers\]](#) as indicated:

```
template<class T>
    const T* any_cast(const any* operand) noexcept;
template<class T>
    T* any_cast(any* operand) noexcept;

-9- Returns: If operand != nullptr && operand->type() == typeid(T) && is_object_v<T>,
a pointer to the object contained by operand; otherwise, nullptr.

[...]
```

[2023-06-14 Varna; Jonathan provides improved wording]

[2023-06-14 Varna; Move to Ready]

Poll: 7-0-1

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 22.7.5 [\[any.nonmembers\]](#) as indicated:

```
template<class T>
    const T* any_cast(const any* operand) noexcept;
template<class T>
    T* any_cast(any* operand) noexcept;

-8- Mandates: is_void_v<T> is false.

-9- Returns: If operand != nullptr && operand->type() == typeid(T) is true, a pointer to the object
contained by operand; otherwise, nullptr.

[...]
```

[3431](#). <=> for containers should require three_way_comparable<T> instead of <=>

Section: 24.2.2.4 [\[container.opt.reqmts\]](#) **Status:** [Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2020-04-17 **Last modified:** 2023-06-16

Priority: 2

Discussion:

The precondition for <=> on containers is:

"Either <=> is defined for values of type (possibly const) T, or < is defined for values of type (possibly const) T and < is a total ordering relationship."

I don't think <=> is sufficient, because *synth-three-way* won't use <=> unless *three_way_comparable*<T> is satisfied, which requires *weakly-equality-comparable-with*<T, T> as well as <=>.

So to use <=> I think the type also requires ==, or more precisely, it must satisfy *three_way_comparable*.

The problem becomes clearer with the following example:

```
#include <compare>
#include <vector>

struct X
{
    friend std::strong_ordering operator<=>(X, X) { return std::strong_ordering::equal; }
};

std::vector<X> v(1);
std::strong_ordering c = v <=> v;
```

This doesn't compile, because despite x meeting the preconditions for <=> in [tab:container.opt], *synth-three-way* will return *std::weak_ordering*.

Here is another example:

```
#include <compare>
#include <vector>

struct X
{
    friend bool operator<(X, X) { return true; } // The return value is intentional, see below
    friend std::strong_ordering operator<=>(X, X) { return std::strong_ordering::equal; }
};

std::vector<X> v(1);
std::weak_ordering c = v <=> v;
```

This meets the precondition because it defines <=>, but the result of <=> on *vector<X>* will be nonsense, because *synth-three-way* will use *operator<* not *operator<=>* and that defines a broken ordering.

So we're stating a precondition which implies "if you do this, you don't get garbage results" and then we give garbage results anyway.

The proposed resolution is one way to fix that, by tightening the precondition so that it matches what *synth-three-way* actually does.

[2020-04-25 Issue Prioritization]

Priority to 2 after reflector discussion.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4861](#).

- 1. Modify 24.2.2 [\[container.requirements.general\]](#), Table [tab:container.opt], as indicated:

Table 75: Optional container operations [tab:container.opt]

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
...				
a <=> b	<i>synth-three-way-result</i> <value_type>	lexicographical_compare_three_way(a.begin(), a.end(), b.begin(), b.end(), <i>synth-three-way</i>)	<i>Preconditions:</i> Either <=> is defined for values of type (possibly const) T satisfies <i>three way comparable</i> , or < is defined for values of type (possibly const) T and < is a total ordering relationship.	linear
...				

[2022-04-24; Daniel rebases wording on [N4910](#)]

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 24.2.2.4 [\[container.opt.reqmts\]](#) as indicated:

a <=> b

-2- *Result*: *synth-three-way-result*<X::value_type>.

-3- *Preconditions*: Either ~~<=> is defined for values of type (possibly const) T~~ *satisfies three_way_comparable*, or < is defined for values of type (possibly const) T and < is a total ordering relationship.

-4- *Returns*: *lexicographical_compare_three_way*(a.begin(), a.end(), b.begin(), b.end(), *synth-three-way*)

[Note 1: The algorithm *lexicographical_compare_three_way* is defined in Clause 27. — *end note*]

-5- *Complexity*: Linear.

[2023-06-13; Varna]

The group liked the previously suggested wording but would prefer to say "models" instead of "satisfies" in preconditions.

[2023-06-14 Varna; Move to Ready]

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 24.2.2.4 [\[container.opt.reqmts\]](#) as indicated:

a <=> b

-2- *Result*: *synth-three-way-result*<X::value_type>.

-3- *Preconditions*: Either ~~<=> is defined for values of type (possibly const) T~~ *models three_way_comparable*, or < is defined for values of type (possibly const) T and < is a total ordering relationship.

-4- *Returns*: *lexicographical_compare_three_way*(a.begin(), a.end(), b.begin(), b.end(), *synth-three-way*)

[Note 1: The algorithm *lexicographical_compare_three_way* is defined in Clause 27. — *end note*]

-5- *Complexity*: Linear.

[3749](#). `common_iterator` should handle integer-class difference types

Section: 25.5.5 [\[iterators.common\]](#) **Status:** [Ready](#) **Submitter:** Hewill Kang **Opened:** 2022-08-01 **Last modified:** 2023-06-14

Priority: 2

Discussion:

The partial specialization of `iterator_traits` for `common_iterator` is defined in 25.5.5.1 [\[common.iterator\]](#) as

```
template<input_iterator I, class S>
struct iterator_traits<common_iterator<I, S>> {
    using iterator_concept = see below;
    using iterator_category = see below;
    using value_type = iter_value_t<I>;
    using difference_type = iter_difference_t<I>;
    using pointer = see below;
    using reference = iter_reference_t<I>;
};
```

where `difference_type` is defined as `iter_difference_t<I>` and `iterator_category` is defined as at least `input_iterator_tag`. However, when `difference_type` is an integer-class type, `common_iterator` does not satisfy *Cpp17InputIterator*, which makes `iterator_category` incorrectly defined as `input_iterator_tag`.

Since the main purpose of `common_iterator` is to be compatible with the legacy iterator system, which is reflected in its efforts to try to provide the operations required by C++17 iterators even if the underlying iterator does not support it. We should handle this case of difference type

incompatibility as well.

The proposed solution is to provide a C++17 conforming difference type by clamping the integer-class type to `ptrdiff_t`.

Daniel:

The second part of this issue provides an *alternative* resolution for the first part of LWG [3748](#) and solves the casting problem mentioned in LWG [3748](#) as well.

[2022-08-23; Reflector poll]

Set priority to 2 after reflector poll.

"I think `common_iterator` should *reject* iterators with integer-class difference types since it can't possibly achieve the design intent of adapting them to *Cpp17Iterators*."

"I'm not yet convinced that we need to outright reject such uses, but I'm pretty sure that we shouldn't mess with the difference type and that the PR is in the wrong direction."

Previous resolution [SUPERSEDED]:

This wording is relative to [N4910](#).

1. Modify 25.5.5.1 [[common.iterator](#)] as indicated:

[Drafting note: `common_iterator` requires iterator type `I` must model `input_or_output_iterator` which ensures that `iter_difference_t<I>` is a signed-integer-like type. The modification of `common_iterator::operator-` is to ensure that the pair of `common_iterator<I, S>` models `sized_sentinel_for` when `sized_sentinel_for<I, S>` is modeled for iterator type `I` with an integer-class difference type and its sentinel type `S`.]

```
namespace std {
    template<class D>
        requires is_signed_integer_like<D>
        using make_cpp17_diff_t = conditional_t<signed_integral<D>, D, ptrdiff_t>; // exposition only

    template<input_or_output_iterator I, sentinel_for<I> S>
        requires (!same_as<I, S> && copyable<I>)
    class common_iterator {
    public:
        [...]
        template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
            requires sized_sentinel_for<S, I2>
        friend constexpr make_cpp17_diff_t<iter_difference_t<I2>> operator-(
            const common_iterator& x, const common_iterator<I2, S2>& y);
        [...]
    };

    template<class I, class S>
    struct incrementable_traits<common_iterator<I, S>> {
        using difference_type = make_cpp17_diff_t<iter_difference_t<I>>;
    };

    template<input_iterator I, class S>
    struct iterator_traits<common_iterator<I, S>> {
        using iterator_concept = see_below;
        using iterator_category = see_below;
        using value_type = iter_value_t<I>;
        using difference_type = make_cpp17_diff_t<iter_difference_t<I>>;
        using pointer = see_below;
        using reference = iter_reference_t<I>;
    };
}
```

2. Modify 25.5.5.6 [[common.iter.cmp](#)] as indicated:

[Drafting note: If this issue is voted in at the same time as LWG [3748](#), the editor is kindly informed that the changes indicated below supersede those of the before mentioned issues first part.]

```
template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
    requires sized_sentinel_for<S, I2>
```

```
friend constexpr make-cpp17-diff-t<iter_difference_t<I2>> operator-(
    const common_iterator& x, const common_iterator<I2, S2>& y);
```

-5- *Preconditions*: `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each false.

-6- *Returns*: 0 if *i* and *j* are each 1, and otherwise `static cast<make-cpp17-diff-t<iter_difference_t<I2>>>(get<i>(x.v_) - get<j>(y.v_))`, where *i* is `x.v_.index()` and *j* is `y.v_.index()`.

[2023-06-13; Varna; Tomasz provides wording]

[2023-06-14 Varna; Move to Ready]

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 25.5.5.1 [\[common.iterator\]](#) as indicated:

```
namespace std {
    [...]

    template<input_iterator I, class S>
    struct iterator_traits<common_iterator<I, S>> {
        using iterator_concept = see below;
        using iterator_category = see below; // not always present
        using value_type = iter_value_t<I>;
        using difference_type = iter_difference_t<I>;
        using pointer = see below;
        using reference = iter_reference_t<I>;
    };
}
```

2. Modify 25.5.5.2 [\[common.iter.types\]](#) as indicated:

-?- The nested typedef-name `iterator_category` of the specialization of `iterator_traits` for `common_iterator<I, S>` is defined if and only if `iter_difference_t<I>` is an integral type. In that case, `iterator_category` denotes `forward_iterator_tag` if the *qualified-id* `iterator_traits<I>::iterator_category` is valid and denotes a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

-1- The **remaining** nested typedef-names of the specialization of `iterator_traits` for `common_iterator<I, S>` are defined as follows:

(1.1) — `iterator_concept` denotes `forward_iterator_tag` if *I* models `forward_iterator`; otherwise it denotes `input_iterator_tag`.

(1.2) — `iterator_category` denotes `forward_iterator_tag` if the *qualified-id* `iterator_traits<I>::iterator_category` is valid and denotes a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

(1.3) — Let *a* denote an lvalue of type `const common_iterator<I, S>`. If the expression `a.operator->()` is well-formed, then `pointer` denotes `decltype(a.operator->())`. Otherwise, `pointer` denotes `void`.

3892. Incorrect formatting of nested ranges and tuples

Section: 22.14.7.2 [\[format.range.formatter\]](#), 22.14.9 [\[format.tuple\]](#) Status: [Ready](#) Submitter: Victor Zverovich Opened: 2023-02-20 Last modified: 2023-06-16

Priority: 2

View all other [issues](#) in [\[format.range.formatter\]](#).

Discussion:

formatter specializations for ranges and tuples set debug format for underlying element formatters in their parse functions e.g. 22.14.7.2 [\[format.range.formatter\]](#) p9:

```
template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

Effects: Parses the format specifier as a *range-format-spec* and stores the parsed specifiers in **this*. The values of *opening-bracket_*, *closing-bracket_*, and *separator_* are modified if and only if required by the *range-type* or the *n* option, if present. If:

- the *range-type* is neither *s* nor *?s*,
- *underlying_.set_debug_format()* is a valid expression, and
- there is no *range-underlying-spec*,

then calls *underlying_.set_debug_format()*.

However, they don't say anything about calling parse functions of those formatters. As a result, formatting of nested ranges can be incorrect, e.g.

```
std::string s = std::format("{", std::vector<std::vector<std::string>>{{"a, b", "c"}});
```

With the current specification *s* is `[[a, b, c]]` instead of `[[ "a, b", "c" ]]`, i.e. strings in the output are not correctly escaped. The same is true for nested tuples and combinations of tuples and ranges.

The fix approved by LEWG as part of [P2733](#) (which was trying to address a different issue) is to always call `parse` [for underlying formatter](#). Additionally the standard should clarify that *format-spec* cannot start with `'` `}` because that's the implicit assumption in range formatting and what happens when *format-spec* is not present.

[2023-03-22; Reflector poll]

Set priority to 2 after reflector poll.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4928](#).

1. Modify 22.14.2.1 [\[format.string.general\]](#) p1 as indicated:

```
[...]
format-specifier:
    : format-spec
format-spec:
    as specified by the formatter specialization for the argument type; cannot start with }
```

2. Modify 22.14.6.1 [\[formatter.requirements\]](#) as indicated:

-3- Given character type *charT*, output iterator type *Out*, and formatting argument type *T*, in Table 74 [tab:formatter.basic] and Table 75 [tab:formatter]:

```
[...]
```

pc.begin() points to the beginning of the *format-spec* (22.14.2 [\[format.string\]](#)) of the replacement field being formatted in the format string. If *format-spec* is **not present or** empty then either *pc.begin()* == *pc.end()* or **pc.begin()* == `' }`.

3. Modify 22.14.7.2 [\[format.range.formatter\]](#) as indicated:

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

-9- *Effects:* Parses the format specifier *s* as a *range-format-spec* and stores the parsed specifiers in **this*. ***Calls *underlying_.parse(ctx)* to parse *format-spec* in *range-format-spec* or, if the latter is not present, empty *format-spec*.*** The values of *opening-bracket_*, *closing-bracket_*, and *separator_* are modified if and only if required by the *range-type* or the *n* option, if present. If:

- (9.1) — the *range-type* is neither *s* nor *?s*,
- (9.2) — *underlying_.set_debug_format()* is a valid expression, and
- (9.3) — there is no *range-underlying-spec*,

then calls *underlying_.set_debug_format()*.

4. Modify 22.14.9 [\[format.tuple\]](#) as indicated:

```
template<class ParseContext>
constexpr typename ParseContext::iterator
```

```
parse(ParseContext& ctx);
```

-7- *Effects*: Parses the format specifier `s` as a *tuple-format-spec* and stores the parsed specifiers in `*this`. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *tuple-type*, if present. For each element `e` in `underlying_`, `calls e.parse(ctx) to parse empty format-spec and`, if `e.set_debug_format()` is a valid expression, calls `e.set_debug_format()`.

[Varna 2023-06-16; Jonathan provides tweaked wording]

Add "an" in two places.

[Varna 2023-06-16; Move to Ready]

This would allow resolving LWG [3776](#) as NAD.

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 22.14.2.1 [\[format.string.general\]](#) p1 as indicated:

```
[...]
format-specifier:
    : format-spec
format-spec:
    as specified by the formatter specialization for the argument type; cannot start with ?.
```

2. Modify 22.14.6.1 [\[formatter.requirements\]](#) as indicated:

-3- Given character type `charT`, output iterator type `Out`, and formatting argument type `T`, in Table 74 [tab:formatter.basic] and Table 75 [tab:formatter]:

```
[...]
```

`pc.begin()` points to the beginning of the *format-spec* (22.14.2 [\[format.string\]](#)) of the replacement field being formatted in the format string. If *format-spec* is `not present or` empty then either `pc.begin() == pc.end()` or `*pc.begin() == '}'`.

3. Modify 22.14.7.2 [\[format.range.formatter\]](#) as indicated:

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

-9- *Effects*: Parses the format specifier `s` as a *range-format-spec* and stores the parsed specifiers in `*this`. `Calls underlying_.parse(ctx) to parse format-spec in range-format-spec or, if the latter is not present, an empty format-spec`. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *range-type* or the *n* option, if present. If:

(9.1) — the *range-type* is neither *s* nor *?s*,

(9.2) — `underlying_.set_debug_format()` is a valid expression, and

(9.3) — there is no *range-underlying-spec*,

then calls `underlying_.set_debug_format()`.

4. Modify 22.14.9 [\[format.tuple\]](#) as indicated:

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

-7- *Effects*: Parses the format specifier `s` as a *tuple-format-spec* and stores the parsed specifiers in `*this`. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *tuple-type*, if present. For each element `e` in `underlying_`, `calls e.parse(ctx) to parse an empty format-spec and`, if `e.set_debug_format()` is a valid expression, calls `e.set_debug_format()`.

[3897](#). `inout_ptr` will not update raw pointer to 0

Section: 20.3.4.3 [\[inout.ptr.t\]](#) Status: [Ready](#) Submitter: Doug Cook Opened: 2023-02-27 Last modified: 2023-06-16

Priority: 2

View all other [issues](#) in [inout.ptr.t].

Discussion:

inout_ptr seems useful for two purposes:

1. Using smart pointers with C-style APIs.
2. Annotating raw pointers for use with C-style APIs.

Unfortunately, as presently specified, it is not safe for developers to use inout_ptr for the second purpose. It is not safe to change code from

```
void* raw_ptr1;
InitSomething(&raw_ptr1);
UpdateSomething(&raw_ptr1); // In some cases may set raw_ptr1 = nullptr.
CleanupSomething(raw_ptr1);
```

to

```
void* raw_ptr2;
InitSomething(std::out_ptr(raw_ptr2));
UpdateSomething(std::inout_ptr(raw_ptr2)); // May leave dangling pointer
CleanupSomething(raw_ptr2);               // Possible double-delete
```

In the case where UpdateSomething would set raw_ptr1 = nullptr, the currently-specified inout_ptr implementation will leave raw_ptr2 at its old value. This would likely lead to a double-delete in CleanupSomething.

The issue occurs because inout_ptr is specified as follows:

1. Constructor: *If the user's pointer is a smart pointer*, perform a "release" operation.
2. (C-style API executes)
3. *If the C-style API returns a non-NULL pointer*, propagate the returned value to the user's pointer.

If the user's pointer is not a smart pointer, no "release" operation occurs, and if the C-style API returns a NULL pointer, no propagation of the NULL occurs. We're left with a dangling raw pointer which is different from the original behavior using &.

I see two potential solutions:

1. Make the "release" operation unconditional (i.e. it applies to both smart and raw pointers). For raw pointers, define the "release" operation as setting the raw pointer to nullptr.
2. Make the return value propagation unconditional for raw pointers.

Solution #2 seems likely to lead to more optimal code as it avoids an unnecessary branch.

[2023-03-22; Reflector poll]

Set priority to 2 after reflector poll.

[Varna 2023-06-16; Move to Ready]

Proposed resolution:

This wording is relative to [N4928](#).

1. Modify 20.3.4.3 [inout.ptr.t] as indicated:

```
~inout_ptr_t();
```

-9- Let SP be *POINTER_OF_OR*(Smart, Pointer) (20.2.1 [memory.general]).

-10- Let *release-statement* be `s.release();` if an implementation does not call `s.release()` in the constructor. Otherwise, it is empty.

-11- *Effects*: Equivalent to:

(11.1) —

```
if (p) {
    apply([&](auto&&... args) {
        s = Smart( static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if is_pointer_v<Smart> is true;

(11.2) — otherwise,

```
release-statement;
if (p) {
    apply([&](auto&&... args) {
        s.reset(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if the expression `s.reset(static_cast<SP>(p), std::forward<Args>(args)...) is well-formed;`

(11.3) — otherwise,

```
release-statement;
if (p) {
    apply([&](auto&&... args) {
        s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));
}
```

if is_constructible_v<Smart, SP, Args...> is true;

(11.4) — otherwise, the program is ill-formed.

3946. The definition of `const_iterator_t` should be reworked

Section: 26.2 [\[ranges.syn\]](#) **Status:** [Ready](#) **Submitter:** Christopher Di Bella **Opened:** 2023-06-13 **Last modified:** 2023-06-26

Priority: Not Prioritized

View other [active issues](#) in `[ranges.syn]`.

View all other [issues](#) in `[ranges.syn]`.

Discussion:

During the reflector discussion of [P2836](#), consensus was reached that `const_iterator_t<R>` doesn't necessarily provide the same type as `decltype(ranges::cbegin(r))`, and that it should be changed to the proposed resolution below so that they're consistent.

[Varna 2023-06-14; Move to Ready]

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 26.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
[...]
template<range R>
    using const_iterator_t = decltype\(ranges::cbegin\(declval<R>\(\)\).const\_iterator<iterator\_t<R>>\); // freestanding
template<range R>
    using const_sentinel_t = decltype\(ranges::cend\(declval<R>\(\)\).const\_sentinel<sentinel\_t<R>>\); // freestanding
[...]
```

Tentatively Ready Issues

[3809](#). Is `std::subtract_with_carry_engine<uint16_t>` supposed to work?

Section: 28.5.4.4 [\[rand.eng.sub\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2022-11-02 **Last modified:** 2023-11-07

Priority: 3

View all other [issues](#) in `[rand.eng.sub]`.

Discussion:

The standard requires `subtract_with_carry_engine<T>` to use:

```
linear_congruential_engine<T, 40014u, 0u, 2147483563u>
```

where each of those values is converted to `T`.

This appears to mean `subtract_with_carry_engine` cannot be used with `uint16_t`, because `2147483563u` cannot be converted to `uint16_t` without narrowing.

What is the intention here? Should it be ill-formed? Should the seed engine be `linear_congruential_engine<uint_least32_t, ...>` instead? The values from the `linear_congruential_engine` are used modulo 2^{32} so getting 64-bit values from it is pointless, and getting 16-bit values from it doesn't compile.

[Kona 2022-11-12; Set priority to 3]

[Kona 2022-11-25; Jonathan provides wording]

[2023-05; reflector poll]

Status to Tentatively Ready after six votes in favour.

Proposed resolution:

This wording is relative to [N4917](#).

1. Modify the class synopsis in 28.5.4.4 [\[rand.eng.sub\]](#) as indicated:

```
namespace std {
    template<class UIntType, size_t w, size_t s, size_t r>
    class subtract_with_carry_engine {
    public:
        // types
        using result_type = UIntType;
        // engine characteristics
        static constexpr size_t word_size = w;
        static constexpr size_t short_lag = s;
        static constexpr size_t long_lag = r;
        static constexpr result_type min() { return 0; }
        static constexpr result_type max() { return m - 1; }
        static constexpr result_type uint_least32_t default_seed = 19780503u;
        // constructors and seeding functions
        subtract_with_carry_engine() : subtract_with_carry_engine(default_seed0u) {}
        explicit subtract_with_carry_engine(result_type value);
        template<class Sseq> explicit subtract_with_carry_engine(Sseq& q);
        void seed(result_type value = default_seed0u);
        template<class Sseq> void seed(Sseq& q);
    };
};
```

2. Modify 28.5.4.4 [\[rand.eng.sub\]](#) p7 as indicated:

```
explicit subtract_with_carry_engine(result_type value);
```

-7- *Effects*: Sets the values of $X_r, \dots, X_{-1}$, in that order, as specified below. If X_{-1} is then 0, sets c to 1; otherwise sets c to 0.

To set the values X_k , first construct e , a `linear_congruential_engine` object, as if by the following definition:

```
linear_congruential_engine<result_typeuint_least32_t,
    40014u, 0u, 2147483563u> e(value == 0u ? default_seed : value);
```

Then, to set each X_k , obtain new values $z_0, \dots, z_{n-1}$ from $n = \lceil w/32 \rceil$ successive invocations of e . Set X_k to $(\sum_{j=0}^{n-1} z_j \cdot 2^{32j}) \bmod m$.

3947. Unexpected constraints on `adjacent_transform_view::base()`

Section: 26.7.27.2 [\[range.adjacent.transform.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Bo Persson **Opened:** 2023-06-17 **Last modified:** 2023-10-27

Priority: Not Prioritized

Discussion:

In section 26.7.27.2 [\[range.adjacent.transform.view\]](#) the class `ranges::adjacent_transform_view` got two new `base()` members from [3848](#).

The first one looks like

```
constexpr V base() const & requires copy_constructible<InnerView>
{ return inner_.base(); }
```

Here the requirement is that `InnerView` is copy constructible, when it in fact returns an object of type `V`. That seems odd.

I would expect the constraint to instead be `copy_constructible<V>`.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 26.7.27.2 [\[range.adjacent.transform.view\]](#), class template `adjacent_transform_view` synopsis, as indicated:

```
namespace std::ranges {
    template<forward_range V, move_constructible F, size_t N>
        requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can_reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
        class adjacent_transform_view : public view_interface<adjacent_transform_view<V, F, N>> {
            [...]
            adjacent_view<V, N> inner_; // exposition only

            using InnerView = adjacent_view<V, N>; // exposition only
            [...]
        public:
            [...]
            constexpr V base() const & requires copy_constructible<InnerView> { return inner_.base(); }
            constexpr V base() && { return std::move(inner_).base(); }
            [...]
        };
}
```

[3948](#). *possibly-const-range* and *as-const-pointer* should be `noexcept`

Section: 26.2 [\[ranges.syn\]](#), 26.3.14 [\[range.prim.cdata\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-06-20 **Last modified:** 2023-10-27

Priority: Not Prioritized

View other [active issues](#) in [\[ranges.syn\]](#).

View all other [issues](#) in [\[ranges.syn\]](#).

Discussion:

As of [P2278R4](#), several range access CPOs are specified with *possibly-const-range* and *as-const-pointer*. These helper functions never throw exceptions, but are not marked with `noexcept`. As a result, implementations are currently allowed to make a call to `ranges::ccpo` potentially throwing while the underlying `ranges::cpo` call is non-throwing, which doesn't seem to be intended.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 26.2 [\[ranges.syn\]](#), header `<ranges>` synopsis, as indicated:

```
[...]
// 26.7.21 \[range.as.const\], as const view
template<input_range R>
    constexpr auto& possibly-const-range(R& r) noexcept { // exposition only
        if constexpr (constant_range<const R> && !constant_range<R>) {
            return const_cast<const R&>(r);
        } else {
            return r;
        }
    }
[...]
```

2. Modify 26.3.14 [\[range.prim.cdata\]](#) before p1 as indicated:

```
template<class T>
    constexpr auto as-const-pointer(const T* p) noexcept { return p; } // exposition only
```

[3949](#). `std::atomic<bool>`'s trivial destructor dropped in C++17 spec wording

Section: 33.5.8.1 [\[atomics.types.generic.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jeremy Hurwitz **Opened:** 2023-06-20 **Last modified:** 2023-10-27

Priority: Not Prioritized

Discussion:

`std::atomic<bool>` was originally required to have a trivial default constructor and a trivial destructor [C++11 [N3337](#): Section [atomics.types.generic], Paragraph 5], the same as the integral [C++11 [N3337](#): Section [atomics.types.generic], Paragraph 5] and pointer specializations [C++11 [N3337](#): Section [atomics.types.generic], Paragraph 6]. [P0558](#) rearranged the text, accidentally (as far as we can tell) removing the constructor and destructor requirements from `std::atomic<bool>`, which has the surprising effect that `std::atomic<bool>` has no longer the same constructor/destructor guarantees as `std::atomic<int>`.

C++20 removed the "trivial default constructor" requirement from all specializations. A specialization for floating-point types was added with a trivial destructor [[N4861](#): Section [atomics.types.float], Paragraph 2)].

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 33.5.8.1 [\[atomics.types.generic.general\]](#) as indicated:

-1- [...]

-2- The specialization `atomic<bool>` is a standard-layout struct. **It has a trivial destructor.**

[3951](#). §[expected.object.swap]: Using `value()` instead of `has_value()`

Section: 22.8.6.5 [\[expected.object.swap\]](#), 22.8.7.5 [\[expected.void.swap\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Ben Craig **Opened:** 2023-06-25 **Last modified:** 2023-10-27

Priority: Not Prioritized

Discussion:

22.8.6.5 [\[expected.object.swap\]](#) p2 has the following text in it:

For the case where `rhs.value()` is false and `this->has_value()` is true, equivalent to: [...]

The table preceding that text is a table of `this->has_value()` vs. `rhs.has_value()`. The `rhs.value()` in the text is almost certainly a typo, as a `.value()` call here doesn't make any sense, especially if this is an `expected<non-bool, E>`.

The same issue is there for 22.8.7.5 [\[expected.void.swap\]](#) p2.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 22.8.6.5 [\[expected.object.swap\]](#) as indicated:

`constexpr void swap(expected& rhs) noexcept(see below);`

-1- [...]

-2- *Effects:* See Table 63 [tab:expected.object.swap].

For the case where `rhs.has_value()` is false and `this->has_value()` is true, equivalent to: [...]

2. Modify 22.8.7.5 [\[expected.void.swap\]](#) as indicated:

`constexpr void swap(expected& rhs) noexcept(see below);`

-1- [...]

-2- *Effects:* See Table 64 [tab:expected.void.swap].

For the case where `rhs.has_value()` is false and `this->has_value()` is true, equivalent to: [...]

3953. `iter_move` for `common_iterator` and `counted_iterator` should return `decltype(auto)`

Section: 25.5.5.7 [common.iter.cust], 25.5.7.7 [counted.iter.cust] **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-06-30 **Last modified:** 2023-11-07

Priority: Not Prioritized

Discussion:

Although the customized `iter_move` of both requires the underlying iterator `I` to be `input_iterator`, they still explicitly specify the return type as `iter_rvalue_reference_t<I>`, which makes it always instantiated.

From the point of view that its validity is only specified in the `input_iterator` concept, it would be better to remove such unnecessary type instantiation, which does not make much sense for an `output_iterator` even if it is still valid.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 25.5.5.1 [common.iterator], class template `common_iterator` synopsis, as indicated:

```
namespace std {
    template<input_or_output_iterator I, sentinel_for<I> S>
        requires (!same_as<I, S> && copyable<I>)
        class common_iterator {
        public:
            [...]
            friend constexpr iter_rvalue_reference_t<I> decltype(auto) iter_move(const common_iterator& i)
                noexcept(noexcept(ranges::iter_move(declval<const I&>())))
                requires input_iterator<I>;
            [...]
        };
    [...]
}
```

2. Modify 25.5.5.7 [common.iter.cust] as indicated:

```
friend constexpr iter_rvalue_reference_t<I> decltype(auto) iter_move(const common_iterator& i)
    noexcept(noexcept(ranges::iter_move(declval<const I&>())))
    requires input_iterator<I>;
```

-1- *Preconditions:* [...]

-2- *Effects:* Equivalent to: `return ranges::iter_move(get<I>(i.v_));`

3. Modify 25.5.7.1 [counted.iterator], class template `counted_iterator` synopsis, as indicated:

```
namespace std {
    template<input_or_output_iterator I>
    class counted_iterator {
    public:
        [...]
        friend constexpr iter_rvalue_reference_t<I> decltype(auto) iter_move(const counted_iterator& i)
            noexcept(noexcept(ranges::iter_move(i.current)))
            requires input_iterator<I>;
        [...]
    };
    [...]
}
```

4. Modify 25.5.7.7 [counted.iter.cust] as indicated:

```
friend constexpr iter_rvalue_reference_t<I> decltype(auto)
    iter_move(const counted_iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current)))
    requires input_iterator<I>;
```

-1- *Preconditions:* [...]

-2- *Effects:* Equivalent to: `return ranges::iter_move(i.current);`

3957. §[container.alloc.reqmts] The value category of v should be claimed

Section: 24.2.2.5 [container.alloc.reqmts] Status: [Tentatively Ready](#) Submitter: jim x Opened: 2023-07-10 Last modified: 2023-10-27

Priority: Not Prioritized

View other [active issues](#) in [container.alloc.reqmts].

View all other [issues](#) in [container.alloc.reqmts].

Discussion:

24.2.2.5 [container.alloc.reqmts] p2 says:

[...] an expression v of type τ or `const  $\tau$` , [...]

Then 24.2.2.5 [container.alloc.reqmts] bullet (2.4) says:

τ is *Cpp17CopyInsertable into x* means that, in addition to τ being *Cpp17MoveInsertable into x* , the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, v)
```

So, what is the value category of the expression v ? We didn't explicitly phrase the wording. The intent may be that the value category of v is any defined value category in 7.2.1 [basic.lval], however, the intent is not clear in the current wording. Maybe, we can say:

[...] the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, v)
```

for v of any value category.

which can make the intent meaning clearer.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#).

1. Modify 24.2.2.5 [container.alloc.reqmts] as indicated:

-2- Given an allocator type A and given a container type x having a `value_type` identical to τ and an `allocator_type` identical to `allocator_traits<A>::rebind_alloc< $\tau$ >` and given an lvalue m of type A , a pointer p of type τ^* , an expression v **that denotes an lvalue** of type τ or `const  $\tau$` **or an rvalue of type `const  $\tau$`** , and an rvalue rv of type τ , the following terms are defined.
[...]

[...]

(2.3) — τ is *Cpp17MoveInsertable into x* means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, rv)
```

and its evaluation causes the following postcondition to hold: The value of $*p$ is equivalent to the value of rv before the evaluation.

[Note 1: rv remains a valid object. Its state is unspecified — *end note*]

(2.4) — τ is *Cpp17CopyInsertable into x* means that, in addition to τ being *Cpp17MoveInsertable into x* , the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, v)
```

and its evaluation causes the following postcondition to hold: The value of v is unchanged and is equivalent to $*p$.

[...]

3965. Incorrect example in [format.string.escaped] p3 for formatting of combining characters

Priority: Not Prioritized

Discussion:

The C++23 DIS contains the following example in 22.14.6.4 [\[format.string.escaped\]](#) p3. (This example does not appear in the most recent [N4950](#) WP or on <https://eel.is/c++draft> because the project editor has not yet merged changes needed to support rendering of some of the characters involved).

```
string s6 = format("[{:?}]", "🙄"); // s6 has value: ["\u{200d}🙄\u{fe0f}"]
```

The character to be formatted (🙄) consists of the following sequence of code points in the order presented:

- U+1F937 (SHRUG)
- U+200D (ZERO WIDTH JOINER)
- U+2642 (MALE SIGN)
- U+FE0F (VARIATION SELECTOR-16)

22.14.6.4 [\[format.string.escaped\]](#) bullet 2.2.1 specifies which code points are to be formatted as a `\u{hex-digit-sequence}` escape sequence:

(2.2.1) — If *X* encodes a single character *C*, then:

(2.2.1.1) — If *C* is one of the characters in Table 75 [tab:format.escape.sequences], then the two characters shown as the corresponding escape sequence are appended to *E*.

(2.2.1.2) — Otherwise, if *C* is not U+0020 SPACE and

(2.2.1.2.1) — *CE* is UTF-8, UTF-16, or UTF-32 and *C* corresponds to a Unicode scalar value whose Unicode property `General_Category` has a value in the groups Separator (Z) or Other (C), as described by UAX #44 of the Unicode Standard, or

(2.2.1.2.2) — *CE* is UTF-8, UTF-16, or UTF-32 and *C* corresponds to a Unicode scalar value with the Unicode property `Grapheme_Extend=Yes` as described by UAX #44 of the Unicode Standard and *C* is not immediately preceded in *S* by a character *P* appended to *E* without translation to an escape sequence, or

(2.2.1.2.3) — *CE* is neither UTF-8, UTF-16, nor UTF-32 and *C* is one of an implementation-defined set of separator or non-printable characters

then the sequence `\u{hex-digit-sequence}` is appended to *E*, where *hex-digit-sequence* is the shortest hexadecimal representation of *C* using lower-case hexadecimal digits.

(2.2.1.3) — Otherwise, *C* is appended to *E*.

The example is not consistent with the above specification for the final code point. [U+FE0F](#) is a single character, is not one of the characters in Table 75, is not U+0020, has a `General_Category` of Nonspacing Mark (Mn) which is neither Z nor C, has `Grapheme_Extend=Yes` but the prior character (U+2642) is not formatted as an escape sequence, and is not one of an implementation-defined set of separator or non-printable characters (for the purposes of this example; the example assumes a UTF-8 encoding). Thus, formatting for this character falls to the last bullet point and the character should be appended as is (without translation to an escape sequence). Since this character is a combining character, it should combine with the previous character and thus alter the appearance of U+2642 (thus producing "🙄" instead of "🙄🙄").

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4950](#) plus missing editorial pieces from [P2286R8](#).

1. Modify the example following 22.14.6.4 [\[format.string.escaped\]](#) p3 as indicated:

[Drafting note: The presented example was voted in as part of [P2286R8](#) during the July 2022 Virtual Meeting but is not yet accessible in the most recent working draft [N4950](#).

Note that the final character (🙄) is composed from the two code points U+2642 and U+FE0F.]

```
string s6 = format("[{:?}]", "🙄"); // s6 has value: ["\u{200d}\u{fe0f}","\u{200d}🙄"]
```

[3970](#). §[mdspan.syn] Missing definition of full_extent_t and full_extent

Section: 24.7.3.2 [mdspan.syn] **Status:** [Tentatively Ready](#) **Submitter:** S. B. Tam **Opened:** 2023-08-16 **Last modified:** 2023-10-27

Priority: Not Prioritized

Discussion:

submdspan uses a type called full_extent_t, but there isn't a definition for that type.

It appears that full_extent_t (along with full_extent) was proposed in [P0009](#) before submdspan was moved into its own paper, and its definition failed to be included in [P2630](#) Submdspan.

[2023-10-27; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 24.7.3.2 [mdspan.syn] as indicated:

```
[...]
// 24.7.3.7 [mdspan.submdspan], submdspan creation
template<class OffsetType, class LengthType, class StrideType>
    struct strided_slice;

template<class LayoutMapping>
    struct submdspan_mapping_result;

    struct full_extent_t { explicit full_extent_t() = default; };
    inline constexpr full_extent_t full_extent{};
[...]
```

[3973](#). Monadic operations should be ADL-proof

Section: 22.8.6.7 [expected.object.monadic], 22.5.3.7 [optional.monadic] **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-08-10
Last modified: 2023-09-24

Priority: Not Prioritized

View all other [issues](#) in [expected.object.monadic].

Discussion:

LWG [3938](#) switched to use **this to access the value stored in std::expected. However, as shown in LWG [3969](#), **this can trigger ADL and find an unwanted overload, and thus may caused unintended behavior.

Current implementations behave correctly ([Godbolt link](#)): they don't direct use **this, but use the name of the union member instead.

Moreover, [P2407R5](#) will change the monadic operations of std::optional to use **this, which is also problematic.

[2023-09-19; Wording update]

Several people preferred to replace operator*() by the corresponding union members, so this part of the proposed wording has been adjusted, which is a rather mechanical replacement.

[2023-10-30; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 22.8.6.7 [expected.object.monadic] as indicated:

[Drafting note: Effectively replace all occurrences of **this by val, except for decltype.]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let U be `remove_cvref_t<invoke_result_t<F, decltype(**this(val))>>`.

-2- [...]

-3- [...]

-4- *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), **thisval);
else
    return U(unexpect, error());
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

-5- Let U be `remove_cvref_t<invoke_result_t<F, decltype((std::move(**thisval))>>`.

-6- [...]

-7- [...]

-8- *Effects*: Equivalent to:

```
if (has_value())
    return invoke(std::forward<F>(f), std::move(**thisval));
else
    return U(unexpect, std::move(error()));
```

```
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) const &;
```

-9- Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

-10- *Constraints*: `is_constructible_v<T, decltype(**this(val))>` is true.

-11- [...]

-12- *Effects*: Equivalent to:

```
if (has_value())
    return G(in_place, **thisval);
else
    return invoke(std::forward<F>(f), error());
```

```
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &&;
```

-13- Let G be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

-14- *Constraints*: `is_constructible_v<T, decltype(std::move(**thisval))>` is true.

-15- [...]

-16- *Effects*: Equivalent to:

```
if (has_value())
    return G(in_place, std::move(**thisval));
else
    return invoke(std::forward<F>(f), std::move(error()));
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

-17- Let U be `remove_cvref_t<invoke_result_t<F, decltype(**this(val))>>`.

-18- [...]

-19- *Mandates*: U is a valid value type for expected. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f), **thisval));
```

is well-formed.

-20- *Effects*:

(20.1) — [...]

(20.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose `has_val` member is true and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), **thisval)`.

(20.3) — Otherwise, evaluates `invoke(std::forward<F>(f), **thisval)` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

-21- Let `U` be `remove_cvref_t<invoke_result_t<F, decltype(std::move(**thisval))>>`.

-22- [...]

-23- *Mandates*: `U` is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f), std::move(**thisval)));
```

is well-formed.

-24- *Effects*:

(24.1) — [...]

(24.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose `has_val` member is true and `val` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(**thisval))`.

(24.3) — Otherwise, evaluates `invoke(std::forward<F>(f), std::move(**thisval))` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

-25- Let `G` be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

-26- *Constraints*: `is_constructible_v<T, decltype(**this(val))>` is true.

-27- *Mandates*: [...]

-28- *Returns*: If `has_value()` is true, `expected<T, G>(in_place, **thisval)`; otherwise, an `expected<T, G>` object whose `has_val` member is false and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

-29- Let `G` be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

-30- *Constraints*: `is_constructible_v<T, decltype(std::move(**thisval))>` is true.

-31- *Mandates*: [...]

-32- *Returns*: If `has_value()` is true, `expected<T, G>(in_place, std::move(**thisval))`; otherwise, an `expected<T, G>` object whose `has_val` member is false and `unex` member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

2. Modify 22.5.3.7 [\[optional.monadic\]](#) as indicated:

[Drafting note: Effectively replace all occurrences of `value()` by `*val`.]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

-1- Let `U` be `invoke_result_t<F, decltype(value()*val)>`.

-2- [...]

-3- *Effects*: Equivalent to:

```
if (*this) {
    return invoke(std::forward<F>(f), value()*val);
} else {
    return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

-4- Let U be `invoke_result_t<F, decltype(std::move(value()*val))>`.

-5- [...]

-6- *Effects*: Equivalent to:

```
if (*this) {
    return invoke(std::forward<F>(f), std::move(value()*val));
} else {
    return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

-7- Let U be `remove_cv_t<invoke_result_t<F, decltype(value()*val)>>`.

-8- *Mandates*: U is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), value()*val));
```

is well-formed for some invented variable u .

[...]

-9- *Returns*: If $*this$ contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), value()*val)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

-10- Let U be `remove_cv_t<invoke_result_t<F, decltype(std::move(value()*val))>>`.

-11- *Mandates*: U is a non-array object type other than `in_place_t` or `nullopt_t`. The declaration

```
U u(invoke(std::forward<F>(f), std::move(value()*val)));
```

is well-formed for some invented variable u .

[...]

-12- *Returns*: If $*this$ contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(value()*val))`; otherwise, `optional<U>()`.

3974. `mdspan::operator[]` should not copy `OtherIndexTypes`

Section: 24.7.3.6.3 [[mdspan.mdspan.members](#)] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2023-08-12 **Last modified:** 2023-11-03

Priority: Not Prioritized

Discussion:

The wording for `mdspan`'s `operator[]` overloads that accept `span` and `array` is in 24.7.3.6.3 [[mdspan.mdspan.members](#)] paragraphs 5 and 6:

```
template<class OtherIndexType>
constexpr reference operator[](span<OtherIndexType, rank()> indices) const;
template<class OtherIndexType>
constexpr reference operator[](const array<OtherIndexType, rank()>& indices) const;
```

-5- *Constraints*:

(5.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true, and

(5.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is true.

-6- *Effects*: Let P be a parameter pack such that

```
is_same_v<make_index_sequence<rank()>, index_sequence<P...>>
```

is true. Equivalent to:

```
return operator[](as_const(indices[P])...);
```

The equivalent code calls the other operator[] overload:

```
template<class... OtherIndexTypes>
constexpr reference operator[](OtherIndexTypes... indices) const;
```

with a pack of const OtherIndexType lvalues, but we notably haven't required OtherIndexTypes to be copyable — we only require that we can convert them to index_type. While one could argue that the use in "Effects: equivalent to" implies a requirement of copyability, it's odd that this implicit requirement would be the only requirement for copyable OtherIndexTypes in the spec. We could fix this by changing the operator[] overload accepting OtherIndexTypes to take them by const&, but that would be inconsistent with virtually every other place in the spec where types convertible to index_type are taken by-value. I think the best localized fix is to perform the conversion to index_type in the "Effects: equivalent to" code so the actual arguments have type index_type which we know is copyable.

[2023-11-02; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 24.7.3.6.3 [\[mdspan.mdspan.members\]](#) as indicated:

```
template<class OtherIndexType>
constexpr reference operator[](span<OtherIndexType, rank()> indices) const;
template<class OtherIndexType>
constexpr reference operator[](const array<OtherIndexType, rank()>& indices) const;
```

-5- Constraints:

(5.1) — is_convertible_v<const OtherIndexType&, index_type> is true, and

(5.2) — is_nothrow_constructible_v<index_type, const OtherIndexType&> is true.

-6- Effects: Let P be a parameter pack such that

is_same_v<make_index_sequence<rank()>, index_sequence<P...>>

is true. Equivalent to:

```
return operator[](extents\_type::index-cast(as_const(indices[P]))....);
```

[3987](#). Including <flat_foo> doesn't provide std::begin/end

Section: 25.7 [\[iterator.range\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-08-27 **Last modified:** 2023-11-03

Priority: Not Prioritized

View other [active issues](#) in [\[iterator.range\]](#).

View all other [issues](#) in [\[iterator.range\]](#).

Discussion:

It seems that 25.7 [\[iterator.range\]](#) should also add <flat_foo> to the list as the latter provides a series of range access member functions such as begin/end.

[2023-11-02; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 25.7 [\[iterator.range\]](#) as indicated:

-1- In addition to being available via inclusion of the <iterator> header, the function templates in [\[iterator.range\]](#) are available when any of the following headers are included: <array>, <deque>, [<flat_map>](#), [<flat_set>](#), <forward_list>, <list>, <map>, <regex>, <set>, , <string>, <string_view>, <unordered_map>, <unordered_set>, and <vector>.

3990. Program-defined specializations of `std::tuple` and `std::variant` can't be properly supported

Section: 22.4.4 [\[tuple.tuple\]](#), 22.6.3.1 [\[variant.variant.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2023-08-29 **Last modified:** 2023-11-03

Priority: Not Prioritized

View all other [issues](#) in [\[tuple.tuple\]](#).

Discussion:

Currently, program-defined specializations of `std::tuple` and `std::variant` are not explicitly disallowed. However, they can't be properly supported by standard library implementations, because the corresponding `std::get` function templates have to inspect the implementation details of these types, and users have no way to make `std::get` behave correctly for a program-defined specializations.

Perhaps we should explicitly disallow specializing `std::tuple` and `std::variant`.

[2023-11-02; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4958](#).

1. Modify 22.4.4 [\[tuple.tuple\]](#) as indicated:

```
namespace std {
    template<class... Types>
    class tuple {
        [...]
    };
    [...]
}
```

-?- If a program declares an explicit or partial specialization of `tuple`, the program is ill-formed, no diagnostic required.

2. Modify 22.6.3.1 [\[variant.variant.general\]](#) as indicated:

[...]

-3- A program that instantiates the definition of `variant` with no template arguments is ill-formed.

-?- If a program declares an explicit or partial specialization of `variant`, the program is ill-formed, no diagnostic required.

4001. `iota_view` should provide empty

Section: 26.6.4.2 [\[range.iota.view\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2023-10-27 **Last modified:** 2023-11-03

Priority: Not Prioritized

View other [active issues](#) in [\[range.iota.view\]](#).

View all other [issues](#) in [\[range.iota.view\]](#).

Discussion:

When `iota_view`'s template parameter is not an integer type and does not model *advanceable*, its `size` member will not be provided as constraints are not satisfied.

If the type further fails to model the *incrementable*, this results in its `view_interface` base being unable to synthesize a valid empty member as `iota_view` will just be an `input_range` ([demo](#)):

```
#include <ranges>
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v;
    auto it = std::back_inserter(v);
    auto s = std::ranges::subrange(it, std::unreachable_sentinel);
    auto r = std::views::iota(it);
    std::cout << s.empty() << "\n"; // 0
}
```

```
std::cout << r.empty() << "\n"; // ill-formed
}
```

This seems to be an oversight. I don't see a reason why `iota_view` doesn't provide `empty` as it does store the start and end like subrange, in which case it's easy to tell if it's empty just by comparing the two.

[2023-11-02; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4964](#).

1. Modify 26.6.4.2 [\[range.iota.view\]](#), class template `iota_view` synopsis, as indicated:

```
namespace std::ranges {
    [...]
    template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>
        requires weakly-equality-comparable-with<W, Bound> && copyable<W>
    class iota_view : public view_interface<iota_view<W, Bound>> {
    private:
        [...]
        W value_ = W();                // exposition only
        Bound bound_ = Bound();        // exposition only
    public:
        [...]
        constexpr iterator begin() const;
        constexpr auto end() const;
        constexpr iterator end() const requires same_as<W, Bound>;

        constexpr bool empty() const;
        constexpr auto size() const requires see below;
    };
    [...]
}

[...]

constexpr iterator end() const requires same_as<W, Bound>;

-14- Effects: Equivalent to: return iterator{bound_};

constexpr bool empty() const;

-?- Effects: Equivalent to: return value == bound;

[...]
```